

SLE-3: Architectural Design (Full C4 Model)

Course: 02AML204 — Introduction to Artificial Intelligence

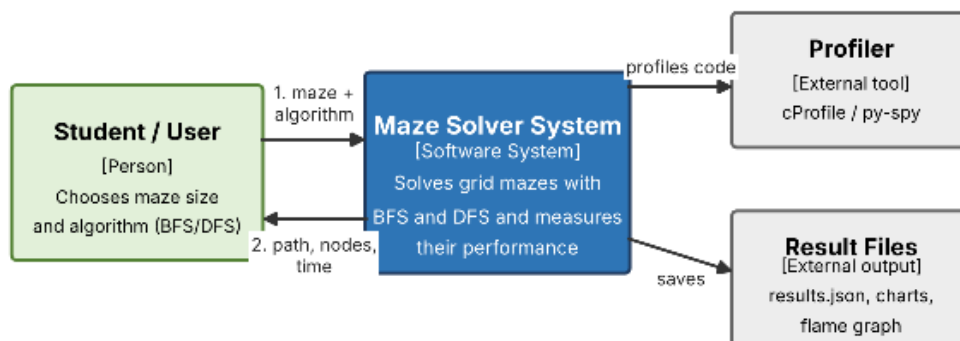
PRN	25UAM130
Name	Mohasin Riyaj Nadaf
Division	B
Date	4 October 2026
GitHub Link	https://github.com/mohsinnadaf45/AL_Augmented_Workflow.git

1. System Title & Short Description

Maze Path-Finding System (BFS & DFS) — continued from SLE-2. It is a Python program that takes a grid maze with walls, one start cell and one goal cell, and finds a path using Breadth-First Search (BFS) or depth-limited Depth-First Search (DFS, limit 1000). For every run it reports the path length, the number of nodes expanded and the execution time. The system solves the path-finding problem and also helps compare the two algorithms: in SLE-2, BFS always returned the shortest path (11, 47 and 134 steps) while DFS returned paths 1.7x–2.2x longer. This report shows the architecture of that system using all 4 levels of the C4 model.

2. Context Diagram (Level 1)

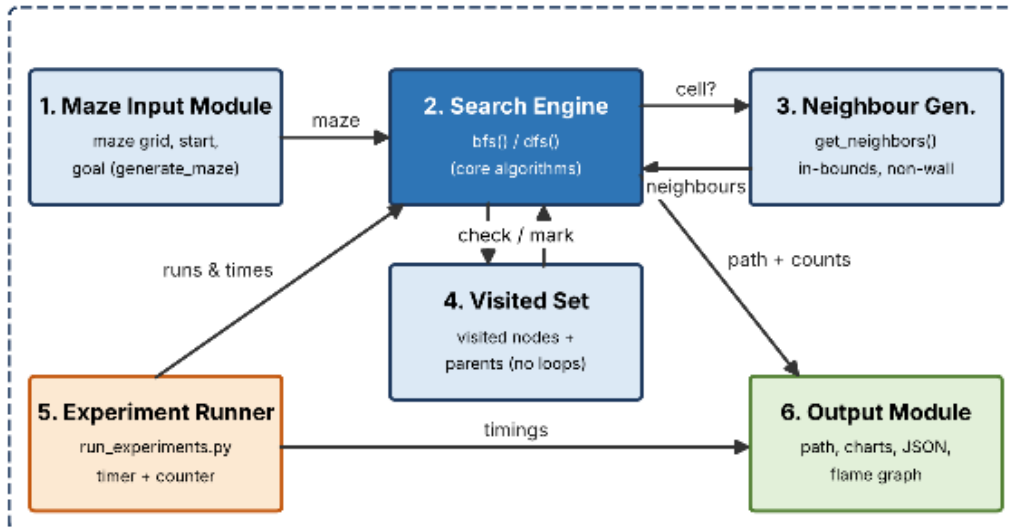
Level 1 - System Context Diagram



The Student (user) gives a maze and chooses BFS or DFS. The Maze Solver System solves it and returns the path, the number of nodes expanded and the time taken. Two outside things are used: a profiler (cProfile; py-spy on a normal machine) that shows where time is spent, and result files (results.json, charts, flame graph) where the output is saved. Everything else is inside one system, so there are no network or database dependencies.

3. Container Diagram (Level 2)

Level 2 - Container Diagram (Maze Solver System)



Explanation of each container:

Maze Input Module: creates or loads the maze grid with walls, start and goal (20×20, 40×40 and 70×70 in SLE-2).

Search Engine: runs bfs() or dfs() and returns (path_length, nodes_expanded, path). This is the core of the system.

Neighbour Generator: get_neighbors() returns the valid, in-bounds, non-wall cells next to a given cell.

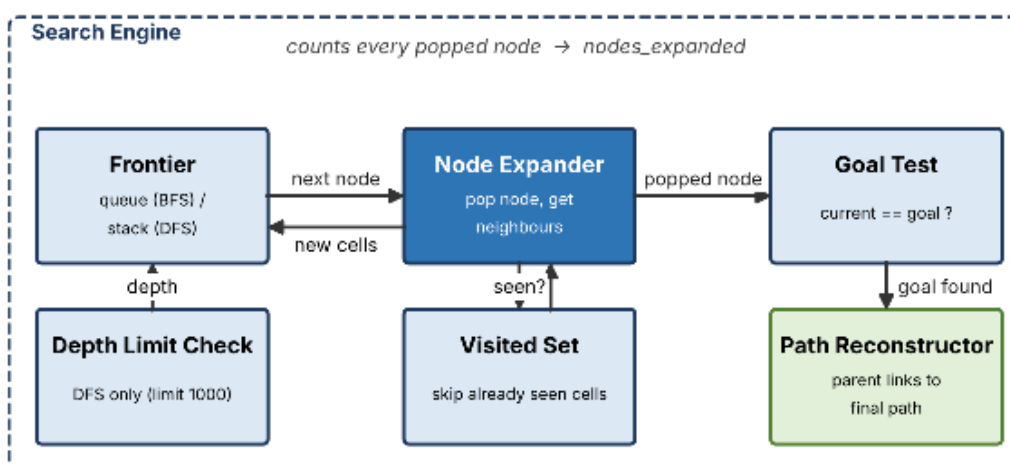
Visited Set / Memory: stores visited cells and their parents so the search never loops and the path can be rebuilt.

Experiment Runner: run_experiments.py runs each algorithm 5 times per maze, times it with time.perf_counter() and collects the node counts.

Output Module: shows the final path and saves results.json, comparison charts and the flame graph.

4. Component Diagram (Level 3)

Level 3 - Component Diagram (inside the Search Engine)



The Search Engine was chosen because it holds all the algorithm logic. The Frontier is a queue for BFS and a stack for DFS — this one difference is what makes the two algorithms behave differently. The Node Expander pops a node (and counts it as expanded), asks for its neighbours,

and adds only unseen ones (checked with the Visited Set) to the Frontier. The Goal Test stops the search when the goal is popped, and the Path Reconstructor follows parent links back to the start to build the path. The Depth Limit Check is used only by DFS (limit 1000).

5. Code Level Overview (Level 4)

def generate_maze(size): builds a grid maze with walls, a start cell and a goal cell.
def get_neighbors(maze, cell): returns valid, in-bounds, non-wall neighbouring cells.
def bfs(maze, start, goal): queue-based search; returns (path_length, nodes_expanded, path); always shortest.
def dfs(maze, start, goal, limit=1000): stack-based search with a depth limit; same return format as bfs().
def reconstruct_path(parents, goal): walks parent links from goal back to start.
run_experiments.py: runs the test mazes, times them with time.perf_counter() and saves results.json.

6. Design Decisions

I split the system into small modules so each has one job: get_neighbors() is separate so BFS and DFS share exactly the same neighbour logic, which makes the SLE-2 comparison fair. bfs() and dfs() return the same (path_length, nodes_expanded, path) tuple, so the Experiment Runner can treat them the same way. The profiling code is kept outside the Search Engine so measuring does not change the algorithm. I kept 6 containers to stay readable, and a Visited Set is used in both searches to avoid infinite loops.

7. AI Contribution Note

AI tools used: Claude (Anthropic) – used to help draft this report and generate the C4 diagram images; GitHub Copilot and Claude were used earlier in SLE-1 and SLE-2 for the code.
What AI helped with: suggesting the C4 layout, drawing the three diagrams with a Python plotting script, and wording the short explanations in the required template.
What I did myself: chose the maze system from my SLE-2 work, decided the container split, picked the Search Engine for the Level 3 view, checked the names and flow against my own bfs(), dfs() and get_neighbors() code and my SLE-2 results, and reviewed the final report.

8. Conclusion

I learned that architecture is easier to understand when it is shown in layers: Context shows who uses the system, Container shows the main parts, Component shows how one part works inside, and Code shows the real functions. Drawing the Search Engine as separate components (Frontier, Visited Set, Goal Test, Path Reconstructor) made it clear that BFS and DFS differ mainly in the Frontier type. It also showed me why separating shared parts like get_neighbors() and the profiling code makes the system easier to test and compare.